Name of the Student	Ch. Venkata Lakshmi Hymavathi
Reg. No	192425397
GitHub Link	

SIMATS ENGINEERING

CO4 - ASSESSMENT TOOL 2

Industry Problem Solving Task

COURSE NAME: Deep Learning **COURSE CODE: MLA0405**
SLOT: D

COURSE OUTCOME COVERED

CO 4: Students will be able to understand and apply probabilistic graphical models including Bayesian Networks, Markov Random Fields, Hidden Markov Models, and Conditional Random Fields. They will learn how to represent complex probabilistic relationships among variables and perform inference and learning in graphical models. Students will be able to use these models for reasoning under uncertainty and solving real-world decision-making problems. BL-6

Course Outcome Weightage: 10% **Duration: 90 minutes**
Assessment Tool Weightage: 40% **Mark : 25**

Code of Conduct:
I **Ch. Venkata Lakshmi Hymavathi (192425397)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
Ch. Hymavathi

Name of the student:
Ch. Venkata Lakshmi Hymavathi

Criteria	Understanding of Industry Problem **(2)**	Application of Engineering Concepts **(2.5)**	Solution Design **(2.5)**	Use of Tools **(2)**	Analysis ***(1)***	**Total (10)**
Weightage	***20 %***	***25%***	***25%***	***20 %***	***10 %***	**100%**
Q1						
Q2						
Q3						
Q4						
Q5						
Total						

QUESTIONS: 05 Total Marks:25

Q1. Transfer Learning

A healthcare startup aims to develop an AI-based system for detecting skin cancer from dermoscopic images. Since only a limited number of labeled images are available, design a transfer learning strategy using a pre-trained CNN model. Justify your choice of model, identify which layers should be frozen or fine-tuned, and explain how your design improves classification performance.

Answer:

The healthcare startup has limited labeled dermoscopic images, so training a CNN from scratch may cause overfitting. **Transfer Learning** can solve this problem by using a CNN already trained on a large image dataset.

Proposed Architecture

Dermoscopic Image
↓
Image Preprocessing
(Resize + Normalize + Augmentation)
↓
Pre-trained CNN
(EfficientNet / ResNet50)
↓
Frozen Convolutional Layers
↓
Fine-tuned Higher Layers
↓
Global Average Pooling
↓
Dense Layer
↓
Dropout
↓
Softmax / Sigmoid
↓
Benign / Malignant

Model Selection

I would choose **EfficientNet** because it provides good classification accuracy while maintaining computational efficiency. A ResNet-based model can also be used because its residual connections make deep feature learning effective.

Transfer Learning Strategy

1. Load a pre-trained CNN such as EfficientNet.
2. Remove its original classification layer.
3. Add:
    - Global Average Pooling
    - Dense layer
    - Dropout
    - Output classification layer.
4. Initially **freeze the convolutional layers**.
5. Train only the newly added classification layers.
6. After initial training, unfreeze the final few CNN layers.

7. Fine-tune them using a **small learning rate**.

Frozen and Fine-Tuned Layers

Layer	**Action**
Initial convolutional layers	Freeze
Middle feature layers	Mostly freeze
Final convolutional blocks	Fine-tune
New classification layers	Train

The early layers learn general features such as edges, textures, and shapes, while deeper layers learn domain-specific patterns. Fine-tuning the final layers allows the network to adapt to skin-lesion characteristics.

Tools

- Python
- TensorFlow/Keras or PyTorch
- OpenCV/PIL
- NumPy
- Scikit-learn
- GPU/Google Colab

Performance Evaluation

Use:

- Accuracy
- Precision
- Recall/Sensitivity
- Specificity
- F1-score
- ROC-AUC
- Confusion matrix

For medical diagnosis, **recall/sensitivity is especially important**, because missing a malignant lesion can have serious consequences.

Conclusion

Transfer learning reduces training time and data requirements while improving generalization. Data augmentation, freezing early layers, and fine-tuning deeper layers can provide an effective skin-cancer classification system. This directly addresses the limited labeled-data constraint described in the question.

Q2. DenseNet and PixelNet

A smart city project requires an automated road-scene analysis system that accurately identifies roads, pedestrians, vehicles, and traffic signs at the pixel level. Design a deep learning architecture using DenseNet and PixelNet to solve this problem. Explain how the proposed architecture improves feature reuse, segmentation accuracy, and computational efficiency.

Answer:

The smart-city system must identify **roads, pedestrians, vehicles, and traffic signs at pixel level**. Therefore, the problem is a **semantic segmentation** problem rather than simple image classification. The assessment specifically asks for a DenseNet and PixelNet-based architecture.

Proposed Architecture

Input Road Image
↓
Preprocessing
↓
DenseNet Backbone
↓
Dense Feature Extraction
↓
Feature Fusion
↓
PixelNet / Segmentation Decoder
↓
Upsampling
↓
Pixel-wise Classification
↓
Segmentation Map
↓
Road | Pedestrian | Vehicle | Sign | Background

Role of DenseNet

DenseNet connects each layer to subsequent layers within a dense block.

Input
↓
Layer 1
↓
Layer 2
↓
Layer 3
↓
Layer 4

This provides:

- Feature reuse
- Better gradient flow
- Reduced redundant feature learning
- Effective deep feature extraction

Role of PixelNet

PixelNet performs pixel-level prediction. The extracted DenseNet features are converted into a segmentation map where every pixel receives a class label.

For example:

Pixel → Class
Road → 0
Pedestrian → 1
Vehicle → 2
Traffic Sign → 3
Background → 4

Advantages

Feature reuse: Dense connections reuse previously learned features.
Segmentation accuracy: Multi-scale features help distinguish objects of different sizes.
Computational efficiency: Dense feature reuse reduces unnecessary feature learning.

Tools

- Python
- TensorFlow/PyTorch
- OpenCV
- GPU
- Cityscapes or similar road-scene dataset

Evaluation

Use:

- Pixel Accuracy
- Mean IoU
- Dice Score
- Precision
- Recall

Conclusion

Combining DenseNet for strong feature extraction with PixelNet for pixel-level prediction produces an efficient road-scene segmentation system capable of identifying roads, pedestrians, vehicles, and traffic signs.

Q3. Bidirectional RNN and Encoder–Decoder

A multinational customer service company wants to build a multilingual chatbot capable of translating customer queries while preserving contextual meaning. Design an Encoder–Decoder sequence-to-sequence model using Bidirectional RNNs. Illustrate the architecture and justify how bidirectional processing improves translation accuracy.

Answer:

The customer-service application requires translation while preserving contextual meaning. An **Encoder–Decoder sequence-to-sequence model using Bidirectional RNNs** is appropriate for this problem.

Architecture

```
Input Sentence
     ↓
Tokenization
     ↓
Word Embeddings
     ↓
Bidirectional RNN Encoder
  ↙        ↘
Forward     Backward
RNN         RNN
  ↘        ↙
Context Representation
     ↓
Attention Mechanism
     ↓
RNN/LSTM Decoder
     ↓
Softmax
     ↓
Translated Sentence
```

Example
Input:
"Where is my order?"

↓

Bidirectional Encoder

↓

Context Vector

↓

Decoder

↓

"¿Dónde está mi pedido?"
Why Bidirectional RNN?
A normal RNN processes information primarily from one direction. A Bidirectional RNN processes the sequence in both directions:

$$ht=[ht;ht]$$

Therefore, each word can use information from both its preceding and succeeding context.
For example:
Customer → service → request
← ← ←
The model obtains richer contextual information.
Encoder
The encoder converts the input sentence into contextual representations.
Words → Embeddings → BiRNN → Context
Decoder
The decoder uses the context representation to generate the translated sentence one word at a time.
is maximized during training.
Attention
An attention mechanism allows the decoder to focus on relevant encoder states rather than relying on a single fixed vector.
Training
Use:
- Parallel multilingual datasets
- Teacher forcing
- Cross-entropy loss
- Backpropagation Through Time

Evaluation
Use:
- BLEU score
- Translation accuracy

- Perplexity
- Human evaluation

Conclusion

Bidirectional processing provides information from both past and future words, resulting in better contextual understanding and improved translation accuracy. The Encoder–Decoder architecture then converts this contextual representation into the target language.

Q4. BPTT and LSTM

A financial analytics company is developing a model to forecast stock prices using historical market data. Create an LSTM-based prediction system and explain how Backpropagation Through Time (BPTT) is used to train the network. Justify why LSTM is more suitable than a traditional RNN for this application.

Answer:

The financial company wants to predict stock prices from historical sequential market data. An **LSTM-based model** is suitable because financial data is sequential and may contain long-term dependencies. The question specifically requires explaining BPTT and why LSTM is preferable to a traditional RNN.

Proposed Architecture

Historical Market Data
(Open, High, Low, Close, Volume)
↓
Preprocessing
Normalization
↓
Time Windows
↓
LSTM Layer
↓
Dropout Layer
↓
LSTM Layer
↓
Dense Layer
↓
Predicted Stock Price

LSTM Cell

An LSTM uses three important gates:

1. **Forget gate**
2. **Input gate**
3. **Output gate**

Forget gate:
Input gate:
Cell state:
Output:

BPTT

Backpropagation Through Time trains the recurrent network by unfolding it over time.

t1 → t2 → t3 → t4 → t5
↓ ↓ ↓ ↓ ↓

LSTM LSTM LSTM LSTM LSTM

↓

Loss

↓

Backward through time

The error is propagated backward through previous time steps to update the network parameters.

Why LSTM Instead of Traditional RNN?

Traditional RNNs can suffer from:

- Vanishing gradients
- Difficulty learning long-term dependencies
- Loss of older information

LSTM uses its memory cell and gates to preserve important information over longer sequences.

Training Process

1. Collect historical market data.
2. Clean missing values.
3. Normalize numerical features.
4. Create time-series windows.
5. Split into training and testing sets chronologically.
6. Train the LSTM using BPTT.
7. Predict future prices.
8. Evaluate predictions.

Evaluation Metrics

Use:

- MAE
- MSE
- RMSE
- MAPE

Conclusion

LSTM is more suitable than a traditional RNN because its gated memory mechanism can preserve useful long-term information and reduce vanishing-gradient problems during BPTT.

Q5. Integrated Deep Learning Solution

A media streaming company plans to automatically generate captions for uploaded videos to improve accessibility. Design an integrated deep learning solution that combines Transfer Learning for visual feature extraction with an LSTM-based Encoder–Decoder architecture for caption generation. Explain the role of each component and justify how the complete system produces accurate captions.

Answer:

The media company wants to automatically generate captions for uploaded videos. The question requires combining **Transfer Learning for visual feature extraction** with an **LSTM Encoder–Decoder** for caption generation.

Complete Architecture

Input Video

↓

Extract Frames

↓

Pre-trained CNN

(Transfer Learning)
↓
Visual Features
↓
Feature Sequence
↓
LSTM Encoder
↓
Context Representation
↓
LSTM Decoder
↓
Word Prediction
↓
Generated Caption

Step 1: Video Processing

The uploaded video is divided into frames.

Video
↓
Frame 1
Frame 2
Frame 3
...
Frame N

Step 2: Transfer Learning

A pre-trained CNN such as **ResNet/EfficientNet** extracts visual features from every frame.

Instead of training the CNN from scratch:

Pre-trained CNN
↓
Freeze early layers
↓
Fine-tune selected layers
↓
Visual feature vectors

Transfer learning reduces the amount of training data and computation required.

Step 3: LSTM Encoder

The sequence of visual features is provided to an LSTM encoder.

Feature 1 → LSTM
Feature 2 → LSTM
Feature 3 → LSTM
Feature 4 → LSTM
↓
Context representation

The encoder learns the temporal information in the video.

Step 4: LSTM Decoder

The decoder generates the caption sequentially.

<START>
↓
"The"

↓
"boy"
↓
"is"
↓
"playing"
↓
"football"
↓
<END>

At each step, the decoder predicts the next word.

Training

The system can be trained using:

- Video-caption datasets
- Pre-trained CNN
- LSTM Encoder–Decoder
- Word embeddings
- Cross-entropy loss
- Teacher forcing

Attention Mechanism

Attention can be added so that the decoder focuses on the most relevant video features while generating each word.

For example:

```
Frame 1 ─┐
Frame 2 ─┤
Frame 3 ─┼──→ Attention → LSTM Decoder → Word
Frame 4 ─┤
Frame 5 ─┘
```

Evaluation

Caption quality can be evaluated using:

- BLEU
- METEOR
- ROUGE
- CIDEr
- Human evaluation

Advantages

Transfer Learning

- Reduces training requirements.
- Provides strong visual features.
- Improves image/video understanding.

LSTM Encoder

- Captures temporal information.
- Understands the sequence of video frames.

LSTM Decoder

- Generates grammatically ordered captions.
- Produces captions word by word.

Example

Input:

Video showing a child playing football

Output:
"A child is playing football."
Conclusion
The integrated system combines the strengths of CNN-based transfer learning and LSTM sequence modeling. The CNN understands **what is present visually**, while the LSTM understands **how the visual events occur over time** and converts them into natural-language captions. This provides an effective solution for automatic video caption generation and accessibility.

RUBRICS FOR CALCULATION:

Criteria	**Weightage**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Developing**	**Level 1 – Beginning**
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	20%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis